



**INSTITUTO POLITÉCNICO
NACIONAL**



ESCUELA SUPERIOR DE CÓMPUTO

BASES DE DATOS

**PRÁCTICA 1: " MODELO ENTIDAD
RELACIÓN"**

**ALUMNA:
MOLINA ZAVALETA ANGIE LALAINÉ**

**PROFESOR:
HURTADO AVILES GABRIEL**

GRUPO: 3BV1

**CIUDAD DE MÉXICO A 17 DE
SEPTIEMBRE DEL 2026**

OBJETIVO

Que usted analice las arquitecturas de los sistemas gestores de bases de datos a partir de su clasificación y de los diversos tipos de bases de datos, construya el entorno de trabajo que empleará durante todo el semestre (control de versiones y un sistema gestor ejecutado en un contenedor), sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda la evidencia se entrega en un repositorio de Git. Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustentan la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado cómo trabajó usted, no solo el resultado final.

EJERCICIO 1. CONTROL DE VERSIONES CON GIT Y GITHUB

Parte A. Investigación. Redacte al menos una cuartilla que responda lo siguiente:

1. ¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?.

Un Sistema de Control de Versiones es como una máquina del tiempo para tu código. Se encarga de guardar un historial detallado de todos los cambios que se le hacen a un proyecto: qué se cambió, quién lo hizo y cuándo.

¿Qué problema resuelve en equipo? Evita el caótico desastre de andar renombrando archivos como `proyecto_final_v2_este_si_bueno.zip`. En un equipo, resuelve el problema de que varias personas trabajen sobre el mismo código al mismo tiempo sin aplastarse ni borrarse el trabajo entre sí. Si alguien comete un error o rompe algo, el equipo puede regresar a una versión funcional previa en segundos.

2. ¿Cuál es la diferencia entre Git y GitHub. No son sinónimos?.

- Git: Es la herramienta local. Es un programa que instalas en tu computadora para rastrear los cambios de tus archivos en tu disco duro. Funciona desde tu terminal sin necesidad de internet.
- GitHub: Es la plataforma en la nube. Es una página web donde subes el historial que creó Git para compartirlo con otros, respaldarlo en la nube y colaborar visualmente en equipo.

3. Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.

- Repositorio: La carpeta principal de tu proyecto que contiene todos tus archivos más todo el historial de cambios guardado por Git.
- Confirmación: Una foto o "punto de guardado" de cómo estaba tu proyecto en un momento exacto, acompañado de un mensaje explicativo.
- Rama: Una línea de tiempo paralela de tu proyecto. Te permite crear una copia del código para experimentar o agregar cosas nuevas sin arriesgar el código principal que ya funciona.
- Fusión: El proceso de tomar los cambios de una rama secundaria e integrarlos a la rama principal cuando ya comprobaste que todo funciona bien.

- Pull Request: Una solicitud formal en plataformas como GitHub donde pides permiso para fusionar tu rama con la principal. Es el espacio donde el equipo revisa tu código, hace comentarios o pide correcciones antes de aprobarlo.
- Archivo .gitignore: Un archivo de texto simple donde le listas a Git qué carpetas o archivos debe ignorar por completo (para no subirlos al repositorio).
- Archivo README.md: La carta de presentación o manual de usuario del proyecto. Es el primer archivo que lee la gente cuando entra al repositorio para entender de qué trata y cómo ejecutarlo.

Flujo de trabajo basado en ramas y revisión entre pares

Consiste en que la rama principal (main) siempre debe estar limpia y libre de errores. Ningún desarrollador programa directamente sobre main. La regla de oro es:

1. Creas una rama individual para cada tarea o feature nueva
2. Trabajas y haces tus commits en tu propia rama.
3. Al terminar, abres un Pull Request para pedir la fusión a main.

¿Por qué se revisa el código entre pares (Code Review)?

- Calidad y prevención de bugs: Un segundo par de ojos detecta errores, fallos de lógica o problemas de seguridad que el autor original pasó por alto.
- Estandarización: Garantiza que todo el equipo siga las mismas reglas de estilo y buenas prácticas de programación.
- Compartir conocimiento: Todo el equipo aprende cómo funcionan las partes del sistema que desarrollaron otros compañeros, evitando que una sola persona sea la única que entienda cómo funciona el código.

PARTE B: PRÁCTICA.

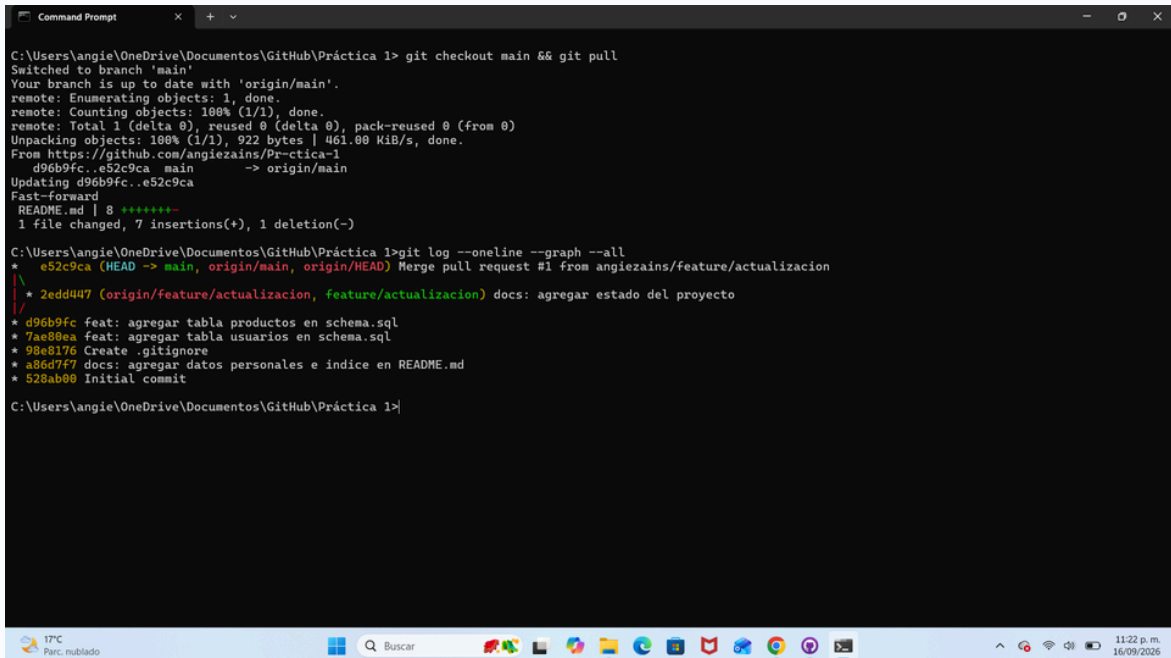
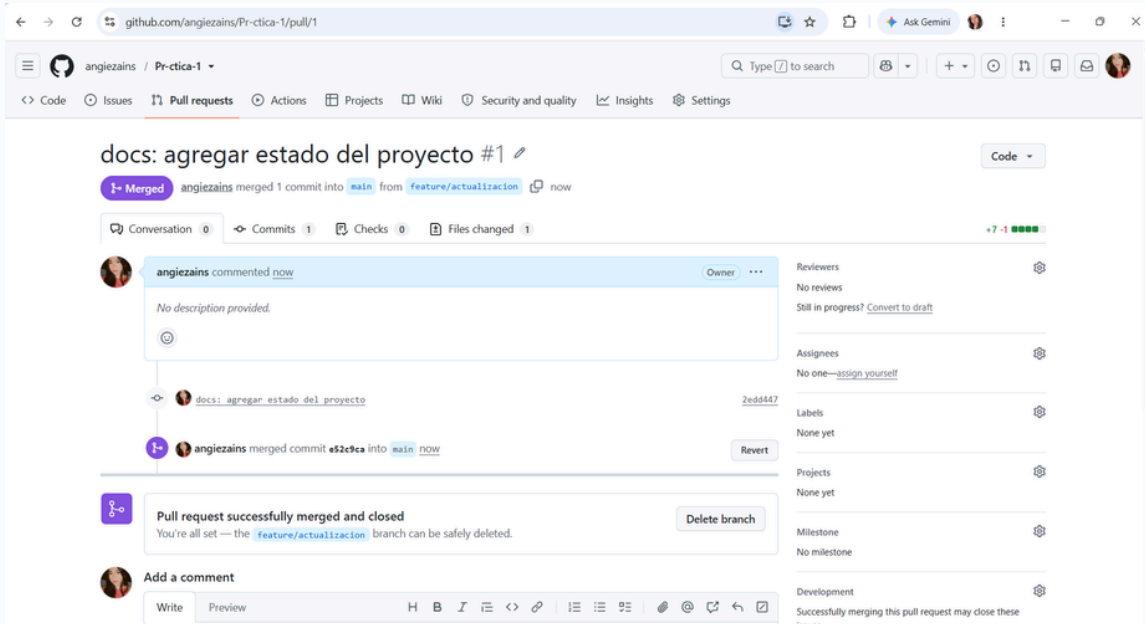
Realice lo siguiente:

1. Cree una cuenta en GitHub, si aún no la tiene, y un repositorio para esta práctica.
2. Agregue un archivo README.md con su nombre completo, número de boleta, grupo, carrera y un índice de la práctica.
3. Agregue un archivo .gitignore adecuado al contenido del repositorio.
4. Realice al menos cinco confirmaciones con mensajes descriptivos. Un mensaje como «cambios» no cuenta.
5. Cree una rama, haga en ella un cambio, abra un Pull Request, descríballo y fusiónelo.

Evidencia: la dirección del repositorio, una captura de la salida de `git log --oneline --graph --all` y una captura del Pull Request ya fusionado.

PARTE B: PRÁCTICA.

Evidencias:



REFERENCIAS

- GitHub: Plataforma de alojamiento de código y gestión de repositorios remotos.
- Git / Git Bash: Sistema de control de versiones distribuido y consola de comandos CLI.
- GitHub Desktop: Interfaz gráfica de usuario (GUI) para la gestión del flujo de trabajo de Git.